

INF05010 OTIMIZAÇÃO COMBINATÓRIA
TRABALHO FINAL: ALGORITMO SIMULATED ANNEALING PARA
O PROBLEMA DE AGENDAMENTO COM DISTÂNCIAS MÍNIMAS

João Vitor Moreira Dias - 00303338

1 Introdução

O objetivo do trabalho consiste em implementar a meta-heurística *Simulated Annealing* para resolver o problema de Agendamento com Distâncias Mínimas. O problema é definido como: dado um conjunto de n tarefas, em que cada tarefa $i \in [n]$ possui um tempo de processamento $p_i \in \mathbb{N}$, determinar os tempos de início $s_i \geq 0$ para todas as tarefas, de modo que $|s_i - s_j| \geq \min p_i, p_j$ para todo par de tarefas $i \neq j \in [n]$, e que o *makespan* $\max_{i \in [n]}(s_i + p_i)$ seja mínimo.

2 Formulação

Variáveis: $s_i \geq 0$, $\forall i \in [n]$, representa o tempo de início da tarefa i , e $t_{i,j} \in \{0, 1\}$, $\forall i < j$, tal que $t_{i,j} = 1$ se a tarefa i é executada antes da tarefa j , e $t_{i,j} = 0$ caso contrário. A variável $y \geq 0$ representa o *makespan* do problema.

Define-se $M = \sum_{i=1}^n p_i$, onde p_i é o tempo de processamento da tarefa i .

Função Objetivo:

$$\min y$$

Restrições:

$$\begin{aligned} y &\geq s_i + p_i && \forall i \in [n] && (1) \\ s_j - s_i &\geq p_i - M(1 - t_{i,j}) && \forall i < j && (2) \\ s_i - s_j &\geq p_j - Mt_{i,j} && \forall i < j && (3) \\ s_i &\geq 0 && \forall i \in [n] && (4) \\ t_{i,j} &\in \{0, 1\} && \forall i < j && (5) \end{aligned}$$

A restrição (1) define um limite superior para *makespan*. As restrições (2) e (3) garantem a condição $|s_i - s_j| \geq \min p_i, p_j$ para todo par de tarefas $i \neq j \in [n]$. Para cada par de

tarefas i e j , a variável binária $t_{i,j}$ indica se p_i é menor que p_j . A restrição (4) impõe a não negatividade dos tempos de início, enquanto a restrição (5) define o domínio binário das variável t .

3 Simulated Annealing

Simulated Annealing é um algoritmo de meta-heurística de busca local introduzido por Kirkpatrick, Gelatt e Vecchi Kirkpatrick et al. (1983). Esse utiliza a ideia de uma temperatura, a qual determina a probabilidade de uma solução pior que a atual ser explorada pela meta-heurística, para evitar mínimos locais. Ao longo da execução, a temperatura decai, e a meta-heurística tende, progressivamente, a não explorar soluções que pioram a função objetivo atual. No trabalho em questão, a condição de parada é definida como número de iterações sem melhoria na solução objetivo, ao invés de uma temperatura mínima, como é usualmente feito na literatura.

4 Parâmetros

Temperatura Inicial (T_0) : temperatura inicial, número real positivo. Controla a probabilidade de aceitação de soluções piores no início do processo.

Taxa de resfriamento (α) : taxa de resfriamento, número real entre 0 e 1. Determina a velocidade em que a temperatura é reduzida a cada iteração.

Máximo de iterações sem melhoria : critério de terminação, número inteiro maior do que 1.

5 O algoritmo

Algorithm 1: Simulated Annealing

Input: Tempos de processamento p , solução inicial $ordemInicial$, limite de tempo

Output: Melhor solução encontrada e respectivo makespan

```
1  $ordemAtual \leftarrow ordemInicial$ ;
2  $makespanAtual \leftarrow CalculaMakespan(ordemAtual)$ ;
3  $melhorOrdem \leftarrow ordemAtual$ ;
4  $melhorMakespan \leftarrow makespanAtual$ ;
5  $T \leftarrow T_0$ ;
6  $iterSemMelhoria \leftarrow 0$ ;
7 while  $iterSemMelhoria < maxIterSemMelhoria$  do
8   for  $k \leftarrow 1$  to  $length(p)$  do
9     Seleciona aleatoriamente duas posições  $a$  e  $b$ ;
10     $makespanNovo \leftarrow AvaliaTroca(ordemAtual, a, b)$ ;
11     $\Delta \leftarrow makespanNovo - makespanAtual$ ;
12     $aceita \leftarrow falso$ ;
13    if  $\Delta < 0$  then
14       $aceita \leftarrow verdadeiro$ ;
15       $iterSemMelhoria \leftarrow 0$ ;
16    else
17       $probabilidade \leftarrow e^{-\Delta/T}$ ;
18      if  $random[0, 1) < probabilidade$  then
19         $aceita \leftarrow verdadeiro$ ;
20    if  $aceita$  then
21       $RealizaTroca(ordemAtual, a, b)$ ;
22       $makespanAtual \leftarrow makespanNovo$ ;
23      if  $makespanAtual < melhorMakespan$  then
24         $melhorOrdem \leftarrow ordemAtual$ ;
25         $melhorMakespan \leftarrow makespanAtual$ ;
26    else
27       $iterSemMelhoria \leftarrow iterSemMelhoria + 1$ ;
28   $T \leftarrow T \times taxaResfriamento$ ;
29 return  $melhorOrdem, melhorMakespan$ ;
```

5.1 Calcula *makespan*

Função utilizada para calcular o *makespan* de uma determinada ordem de tarefas, determinando o menor instante de início possível para cada tarefa com base nas tarefas previamente alocadas.

5.2 Avalia troca

Função utilizada para avaliar o impacto da troca de duas tarefas na solução, verificando se o *makespan* resultante é melhor ou pior em relação ao da solução atual.

6 Implementação

6.1 Plataforma de execução

O trabalho foi implementado e testado em sistema operacional Windows 10 Pro (64-bit), versão 10.0.19045.2006, em um computador com processador AMD FX-8320E Eight-Core Processor, operando a 3.20 GHz, com 4 núcleos físicos e 8 threads lógicas. O sistema possui cache L2 de 2 MB, e 12 GB de memória RAM. A implementação foi realizada utilizando a linguagem Julia (versão 1.12.1), compilada com LLVM 18.1.7, executada em ambiente Windows x86_64.

6.2 Estruturas de dados utilizadas

Para representar uma solução candidata, utilizou-se um vetor de tamanho n , onde cada posição corresponde à ordem de execução das tarefas. Os tempos de processamento das tarefas foram armazenados em um vetor unidimensional também de tamanho n .

7 Testes de parâmetros

Nos testes de calibração dos parâmetros, cada parâmetro foi avaliado individualmente, mantendo-se os demais fixos. Para isso, o teste foi realizado em 4 das 10 instâncias do problema original, sendo estas *adm_100_1*, *adm_100_2*, *adm_1000_1* e *adm_1000_2*. Com os resultados da meta-heurística para as quatro instâncias, calculou-se a otimalidade média e o tempo médio de execução para cada uma das configurações avaliadas.

7.1 Teste do parâmetro *taxa de resfriamento*

A taxa de resfriamento foi avaliada considerando os seguintes valores: (0,10), (0,15), (0,20), (0,25), (0,30), (0,35), (0,40), (0,45), (0,50), (0,55), (0,60), (0,65), (0,70), (0,75), (0,80), (0,85), (0,90), (0,945) e (0,99). A temperatura inicial foi mantida fixa em (100,0), e o número máximo de iterações consecutivas sem melhoria da função objetivo foi definido como (1000). As Figuras 1 e 2 mostram os resultados. É evidente a importância desse parâmetro, já que com isso o modelo tem mais tempo de explorar caminhos que inicialmente não melhoram a função objetivo. Percebe-se que a otimalidade fica constante por volta do valor 0.945, o qual, por esse motivo, foi escolhido para o teste de instâncias.

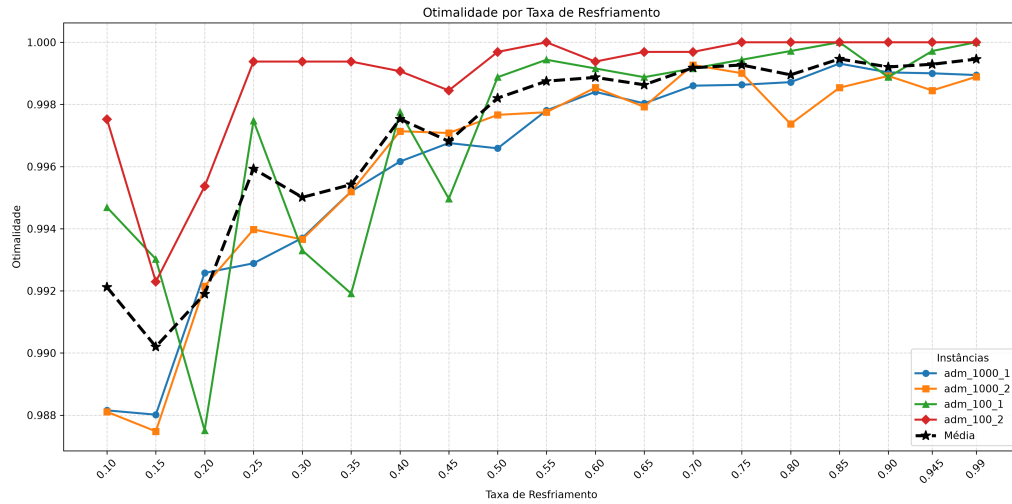


Figura 1: Otimalidade obtida para cada instância em função da taxa de resfriamento.

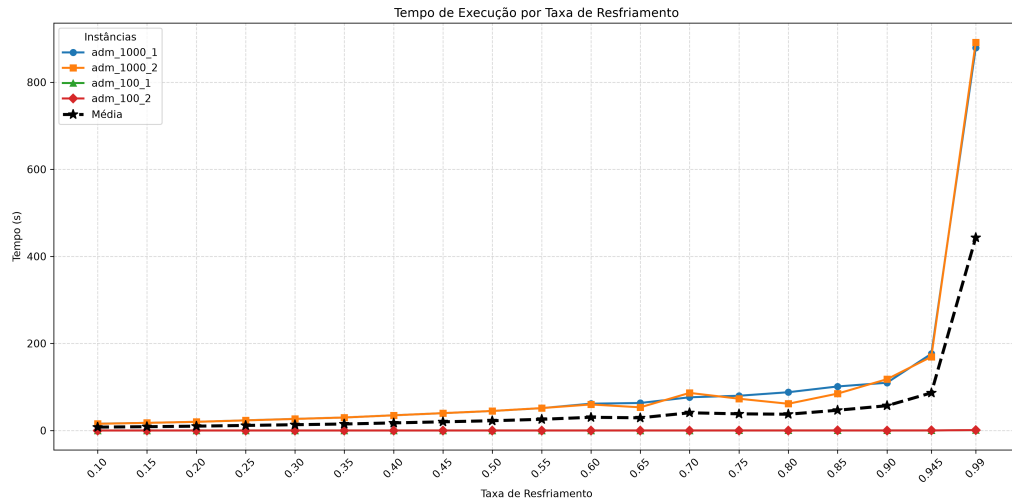


Figura 2: Tempo de execução para cada instância em função da taxa de resfriamento.

7.2 Teste do parâmetro *máximo de iterações sem melhoria*

O parâmetro de número máximo de iterações sem melhoria foi avaliado considerando os seguintes valores: (10), (100), (250), (500), (750), (1000) e (1500). Para todos os experimentos, a temperatura inicial foi mantida fixa em (100,0), enquanto a taxa de resfriamento foi definida como (0,80). As Figuras 3 e 4 mostram os resultados. Percebe-se que aumentar o parâmetro não produz melhores resultados e apenas aumenta o tempo

de computação a partir do valor 750.

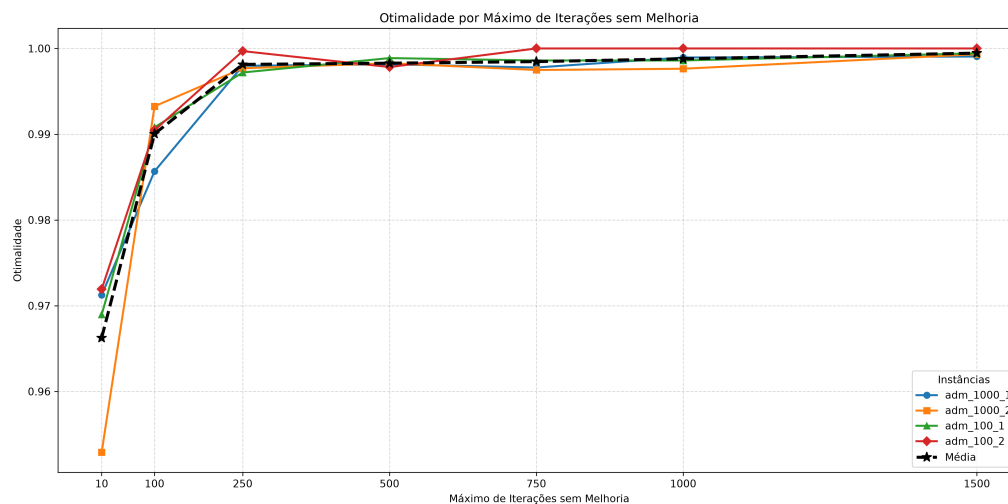


Figura 3: Otimalidade por instância em função do número máximo de iterações sem melhoria.

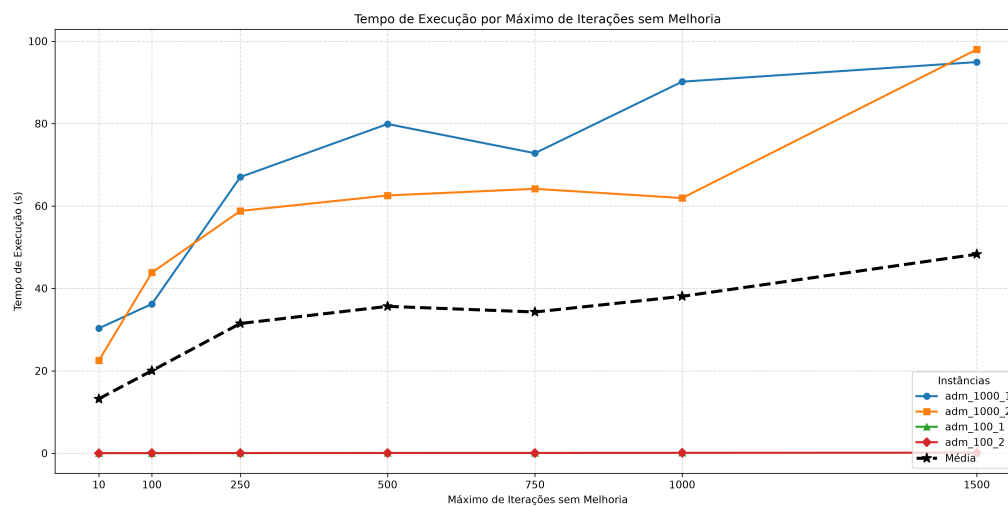


Figura 4: Tempo de execução por instância em função do número máximo de iterações sem melhoria.

7.3 Teste do parâmetro *temperatura inicial*

A temperatura inicial foi avaliada considerando os seguintes valores: (1,0), (5,0), (10,0), (25,0), (50,0), (100,0), (250,0), (500,0), (1000,0) e (2000,0). Para todos os experimentos, a taxa de resfriamento foi mantida fixa em (0,90), e o número máximo de iterações consecutivas sem melhoria foi definido como (500). As Figuras 5 e 6 mostram os resultados. Pelo comportamento da curva de otimalidade, percebe-se que, ao aumentar esse parâmetro, inicialmente a busca produz resultados melhores, pois não fica presa a mínimos locais. Porém, para valores acima de 250, a busca passa a se comportar de forma mais aleatória, reduzindo a otimalidade das soluções obtidas.

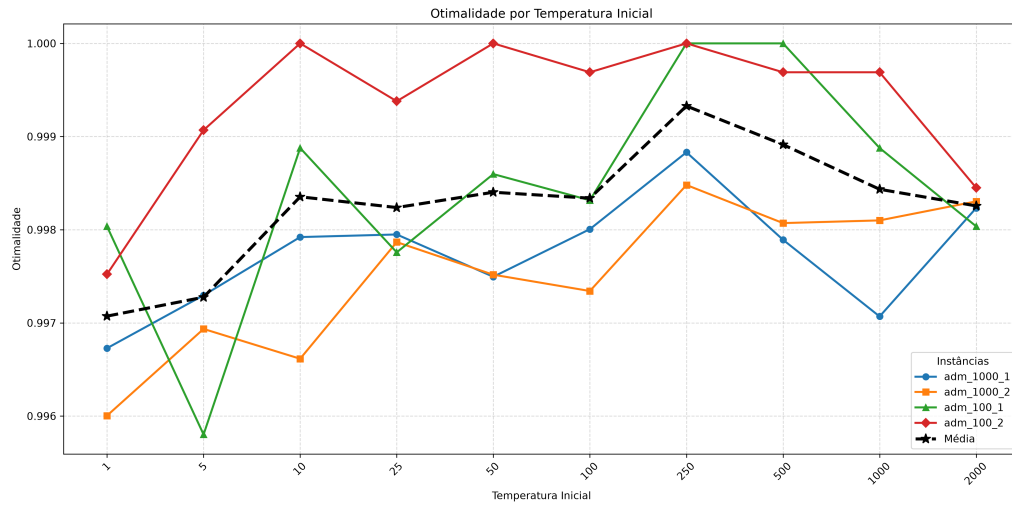


Figura 5: Otimalidade por instância em função da temperatura inicial.

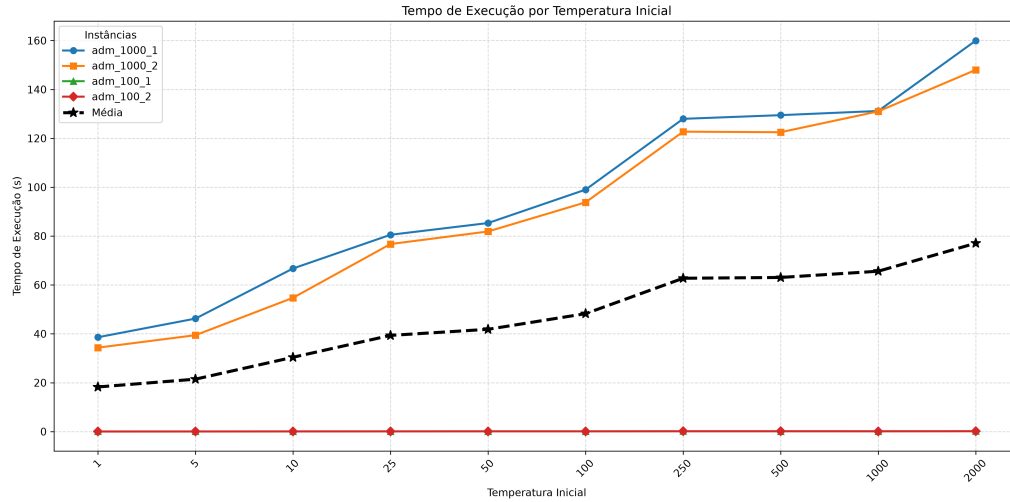


Figura 6: Tempo de execução por instância em função da temperatura inicial.

8 Testes das instâncias

Nos testes de instâncias, foram avaliados o *Simulated Annealing*, a busca gulosa e o *solver* utilizando as instâncias apresentadas na Tabela 1.

Tabela 1: Soluções de referência utilizadas para avaliação das instâncias.

Instância	Solução de Referência	Instância	Solução de Referência
adm_50_1	1906	adm_1000_3	34124
adm_50_2	1888	adm_1000_4	34340
adm_50_3	1532	adm_100_3	3731
adm_50_4	1726	adm_100_4	2792
adm_100_1	3558	adm_1000_1	35035
adm_100_2	3220	adm_1000_2	34148

8.1 Testes das instâncias sem o *annealing*

Para fins de comparação, este teste executa o mesmo procedimento de busca utilizado pelo *Simulated Annealing*, porém com temperatura nula. Dessa forma, o critério probabilístico de aceitação de soluções é desabilitado, fazendo com que o algoritmo aceite apenas soluções que apresentam melhoria, caracterizando uma busca gulosa. Os resultados obtidos são apresentados na Tabela 2. Percebe-se que o algoritmo apresenta um desvio pequeno em relação à solução referência.

Tabela 2: Resultados da busca gulosa em relação às soluções referência.

Instância	Solução Referência	Solução Final	Desvio para a Referência (%)	Tempo Médio (s)
adm_1000_1	35035	35556	1,49	11,78
adm_1000_2	34148	34665	1,51	11,77
adm_1000_3	34124	34492	1,08	11,72
adm_1000_4	34340	34810	1,37	11,76
adm_100_1	3558	3604	1,29	0,01
adm_100_2	3220	3242	0,68	0,02
adm_100_3	3731	3822	2,44	0,01
adm_100_4	2792	2829	1,32	0,01
adm_50_1	1906	1926	1,05	0,00
adm_50_2	1888	1892	0,21	0,00
adm_50_3	1532	1566	2,22	0,00
adm_50_4	1726	1774	2,78	0,00

8.2 Testes das instâncias com o *annealing*

Os resultados apresentados na Tabela 3 referem-se à execução do algoritmo *Simulated Annealing* utilizando os parâmetros definidos durante a etapa de calibração. Observa-se que o método foi capaz de produzir soluções muito próximas às soluções referência em todas as instâncias avaliadas, apresentando desvios inferiores a 0,2

Tabela 3: Qualidade das soluções obtidas pelo algoritmo Simulated Annealing.

Instância	Solução Referência	Valor Inicial	Valor Final Médio	Desvio Padrão	Desvio para a Referência (%)	Desvio para a Sol. Inicial (%)
adm_1000_1	35035	40461,0	35083,8	9,81	0,14	13,29
adm_1000_2	34148	39377,0	34190,4	5,86	0,12	13,17
adm_1000_3	34124	39105,0	34156,0	5,79	0,09	12,66
adm_1000_4	34340	39295,0	34394,2	27,06	0,16	12,47
adm_100_1	3558	4108,0	3559,4	1,95	0,04	13,35
adm_100_2	3220	3652,0	3220,0	0,00	0,00	11,83
adm_100_3	3731	4211,0	3732,4	1,14	0,04	11,37
adm_100_4	2792	3284,0	2793,2	1,10	0,04	14,95
adm_50_1	1906	2131,0	1906,0	0,00	0,00	10,56
adm_50_2	1888	2145,0	1888,0	0,00	0,00	11,98
adm_50_3	1532	1862,0	1533,8	1,30	0,12	17,63
adm_50_4	1726	2045,0	1726,6	0,89	0,03	15,57

Tabela 4: Tempo de execução e número de iterações do algoritmo Simulated Annealing.

Instância	Tempo Médio (s)	Desvio Padrão	Iterações Médias	Desvio Padrão
adm_1000_1	205,68	6,36	120400	3715
adm_1000_2	206,78	6,91	121200	4087
adm_1000_3	204,99	2,56	120400	1517
adm_1000_4	198,15	13,82	116000	8124
adm_100_1	0,22	0,02	13120	776
adm_100_2	0,21	0,02	12520	554
adm_100_3	0,22	0,01	13560	573
adm_100_4	0,23	0,02	13600	851
adm_50_1	0,03	0,01	6880	256
adm_50_2	0,03	0,00	7110	305
adm_50_3	0,03	0,00	6770	368
adm_50_4	0,03	0,01	6650	395

8.3 Execução com o solver HiGH

Os resultados apresentados na Tabela 5 são correspondentes aos testes das instâncias utilizando o *solver* com tempo máximo de 30 minutos. Para os valores de solução iguais a zero, o *solver* não encontrou uma solução inteira factível dentro do tempo máximo de execução. Observa-se que, para instâncias muito grandes, o *solver* torna-se impraticável, pois não consegue alcançar resultados aceitáveis no tempo proposto.

Tabela 5: Resultados da busca gulosa em relação às soluções referência.

Instância	Solução Referência	Valor Obtido	Desvio para Referência (%)	Desvio para Inicial (%)	Solução Inicial	Tempo (s)	Iterações
adm_50_1	1906	1982	3,99	6,99	2131	1500,1	1145000
adm_50_2	1888	1943	2,91	9,42	2145	1579,6	1066000
adm_50_3	1532	1636	6,79	12,14	1862	1771,8	1291000
adm_50_4	1726	1845	6,89	9,78	2045	536,8	530183
adm_100_1	3558	3911	9,92	4,80	4108	1432,8	386420
adm_100_2	3220	3446	7,02	5,64	3652	761,1	380752
adm_100_3	3731	4109	10,13	2,42	4211	1374,7	560558
adm_100_4	2792	3053	9,35	7,95	3284	222,8	49430
adm_1000_1	35035	0	100,00	100,00	0	0	0
adm_1000_2	34148	0	100,00	100,00	0	0	0
adm_1000_3	34124	0	100,00	100,00	0	0	0
adm_1000_4	34340	0	100,00	100,00	0	0	0

8.4 Comparação dos testes das instâncias

A Figura 7 apresenta os resultados obtidos para todas as instâncias avaliadas. Observa-se que a meta-heurística superou o *solver* e a busca gulosa (correspondente ao mesmo algoritmo sem o processo de *annealing*) em todas as instâncias analisadas.

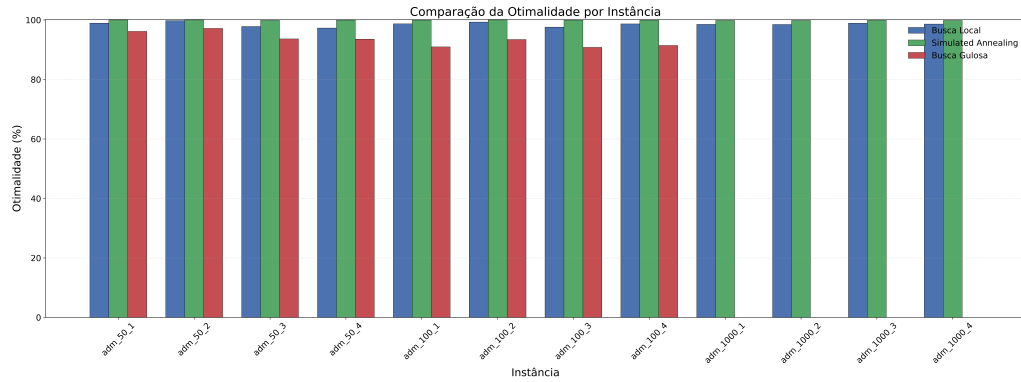


Figura 7: Comparação da otimalidade por instância.

9 Conclusões

O algoritmo apresentou um desempenho muito bom, alcançando um fator de otimalidade de no mínimo 99,84% para as instâncias testadas. O algoritmo superou o *solver* em todos os casos avaliados quanto à melhor solução encontrada, tornando vantajosa a utilização da meta-heurística em relação ao *solver* HiGHS. A execução da meta-heurística com os parâmetros obtidos durante a etapa de calibração apresentou resultados superiores aos da busca gulosa em todos os casos, apesar dessa executar de maneira mais rápida.

O algoritmo também mostrou-se capaz de apresentar soluções muito próximas do melhor valor conhecido em casos nos quais o modelo não apresentou resultados em tempo aceitável. A partir disso, conclui-se que o trabalho foi bem-sucedido e que a meta-heurística utilizada é a melhor alternativa em relação ao *solver* e à busca gulosa para as instâncias estudadas, embora a última tenha obtido resultados mais rapidamente.

Referências

Kirkpatrick, S., Gelatt, C. D., and Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598):671–680.